LAB 7 – SSAS INTRODUCTION

CONTENT

- Data Cubes
    a. Data Dictionary for Adventure Works DW – database after ETL process:
        i. https://dataedo.com/samples/html/Data_warehouse/doc/AdventureWorksDW_4/home.html
    b. SQL Server Analysis Services
        i. https://docs.microsoft.com/en-au/sql/analysis-services/analysis-services
    c. SSAS Multidimensional model
        i. https://docs.microsoft.com/en-au/sql/analysis-services/multidimensional-models/multidimensional-models-ssas
    d. Analysis Services Tutorials - Focus on Multidimensional Model
        i. https://docs.microsoft.com/en-au/sql/analysis-services/multidimensional-modeling-adventure-works-tutorial
        ii. http://www.codeproject.com/Articles/658912/Create-First-OLAP-Cube-in-SQL-Server-Analysis-Serv
        iii. https://www.mssqltips.com/sqlservertutorial/2000/sql-server-analysis-services-ssas/
        iv. https://www.youtube.com/results?search_query=ssas+tutorial
    e. General Resources - https://docs.microsoft.com/en-us/sql/analysis-services/multidimensional-modeling-adventure-works-tutorial
        i. Focus on defining Advanced Attribute and Dimension Properties, hiding and Disabling Attribute Hierarchies, defining the Unknown Member and Null Processing Properties, defining Calculations, defining Perspectives, defining Key Performance Indicators (KPIs)

TASK 1 – Basic cube creation – tutorial

Basic cube creation resources:

- *For a short introduction on how to build a cube please follow the tutorial provided by sqlshack: https://www.sqlshack.com/build-cube-scratch-using-sql-server-analysis-services-ssas/*
- *For detailed description of the process please follow the basic tutorial provided by MS on SQL Server Analysis Services available at https://docs.microsoft.com/en-us/analysis-services/multidimensional-tutorial/multidimensional-modeling-adventure-works-tutorial?view=asallproducts-allversions Focus on Lesson 1, 2 and 3 – further using the acquired knowledge please do the following tasks (please note that we are not directly following the steps described in the lessons).*

1. Investigate the structure of AdventureWorksDW2014 database:
    a. Please focus on the following dimension tables:
        i. DimCustomer (dbo), DimDate (dbo), DimGeography (dbo), DimProduct (dbo), DimProductSubcategory (dbo), DimProductCategory (dbo)
    b. Please focus on the following fact table:
        i. FactInternetSales (dbo)
2. Run Visual Studio, and create a new Analysis Services Project Multidimensional. Within the project Define a data source and a data source view:
    a. Resources: Focus on *Lesson 1: Defining a Data Source View within an Analysis Services Project*: https://docs.microsoft.com/en-us/analysis-services/multidimensional-tutorial/lesson-1-defining-a-data-source-view-within-an-analysis-services-project?view=asallproducts-allversions
    b. Define a new data source using AdventureWorksDW2014 database. Open Solution Explorer (right pane), locate the Data Sources folder and launch the Wizard (right-click the folder and select New Data Source). Further select Create a data source based on an existing or new connection and click New and define the connection. Next, make sure to select proper Impersonation Information (it tells SSAS which credentials to use when processing data – on

laboratory workstations we are using the service account, as it is the easiest for development if the service account has DB permissions).

c. Define a new data source view using the already established data source. Data Source View is a logical layer where you select a subset of tables, define relationships, and add "virtual" columns without altering the physical database. Launch the Wizar by right-clicking the Data Source Views folder in Solution Explorer and select New Data Source View. Select Data Source by picking the Data Source you created in the previous step. Select Tables and Views from the Available objects list (move the tables you need to the Included objects list using the > arrow). Please use the following tables from AdventureWorksDW2014:
   i. Dimension tables - DimDate (dbo), DimCustomer (dbo), DimProduct (dbo),
   ii. Fact table - FactInternetSales (dbo)

d. There are some key actions available within the DSV. Once your DSV is created, you will likely need to refine it before building cubes. You can create logical relationships, especially If your database lacks foreign keys, you can manually drag a column from one table to another in the DSV designer to create a relationship. You can create named calculations (right-click a table and select New Named Calculation) to write a SQL expression (e.g., CONCAT(A1,A2)) that appears as a new column in SSAS. Finally, you can introduce named queries, if you need to filter data or join tables before they hit the model (right-click the background and select New Named Query) - this acts like a "Virtual View" inside your project.

3. Define and deploy a very basic cube:
   a. Resources: *Focus on Lesson 2: Defining and Deploying a Cube:* https://docs.microsoft.com/en-us/analysis-services/multidimensional-tutorial/lesson-2-defining-and-deploying-a-cube?view=asallproducts-allversions
   b. First define the structure of DimDate time dimensions (under Dimensions in Solution Explorer pane – choose new dimension). Create the dimension based on DimDate (dbo) source table. Then:
      i. Select all English names and basic information - DateKey, FullDateAlternateKey, DayNumberOfWeek, EnglishDayNameOfWeek, DayNumberOfMonth, DayNumberOfYear, EnglishMonthName, WeekNumberOfYear, MonthNumberOfYear, CalendarQuarter, CalendarYear, CalendarSemester, FiscalQuarter, FiscalYear, FiscalSemester
   c. Next, define a new cube (under Cubes in Solution Explorer pane – choose new cube). Use existing sources – as we already have source data in our source data view. In the following steps select Fact table and measures, use existing tables to create dimensions (without DimDate – deselect it, as we have already created it beforehand).
      i. Use basic measures from FactInternetSales
         1. use basic measures like OrderQuantity, SalesAmount, and consider adding at least two more – adequate – measures
      ii. Use three dimensions
         1. already created Time dimension (Select Exisiting Dimension step)
         2. two new dimensions Customer and Product (Select New Dimension step) based on DimCustomer and DimProduct tables (deselect DimProductCategory and DimProductSubcategory).
   d. Next, just deploy and process the cube – right click on the newly created cube and select Process. This will first deploy the cube and then fill it with data.

NOTE – If you are having problems with Impersonation Mode follow the basic solution. Impersonation is what happens server side after you deploy project. SSAS then needs to somehow handle the process of actual data movement – from sources to SSAS – and it requires a user to do the actual reading from the data source. As such, impersonation is what SSAS uses to access the source database systems.

> *For more details, see https://docs.microsoft.com/en-us/analysis-services/multidimensional-models/set-impersonation-options-ssas-multidimensional?view=asallproducts-allversions*

Basic solution - use the service account: In general, when you are installing Analysis Services it sets up a service account that Analysis Services uses to run. You can identify this account by going to Start –> Services, looking for SQL Server Analysis Services, and selecting properties – go to the Log On tab. This is the account the SSAS is using (. Setting service account as the impersonation mode, in short, means that you select that the service account will

run the process operation (and runs the heart of Analysis Services). You'd have to give this account access to any data source where SSAS would need to reference data from (e.g., add datareader role). Please understand that this is a strictly development approach, in production systems it's discouraged (due to security reasons).

4. If everything finished successfully, open PowerBI and try connecting to the cube (use Analysis Services as connection type and select live connection – you can check your Analysis Services server name in SQL Server Management Studio – by connecting to an Analysis Services instance and browsing for available servers).

Next, let us modify the cube a bit, as currently it lacks important attributes in dimensions.

5. Modify measures and dimensions.
    a. Resources: *Focus on Lesson 3: Modifying Measures, Attributes and Hierarchies*: https://docs.microsoft.com/en-us/analysis-services/multidimensional-tutorial/lesson-3-modifying-measures-attributes-and-hierarchies?view=asallproducts-allversions
    b. Add attributes to dimensions – you can do that by modifying the structure of dimensions in the Dimensions folder in the Solution Explorer:
        i. Customer
            1. CustomerKey, CustomerAlternateKey, FirstName, MiddleName, LastName, NameStyle, BirthDate, MaritalStatus, Gender, YearlyIncome, TotalChildren, NumberChildrenAtHome, HouseOwnerFlag, NumberCarsOwned, CommuteDistance
        ii. Product
            1. ProductKey, ProductAlternateKey, EnglishProductName, StandardCost, FinishedGoodsFlag, Color, ListPrice, Size, SizeRange, Weight, DaysToManufacture, ProductLine, StartDate, EndDate, Status,
        iii. Note that we have only basic Product information and forgot to include ProductSubcategory and ProductCategory data. To correct this, we need to modify the source data view by adding two missing tables – right click on the empty space between the tables in the source data view (in the source data view part of the solution explorer). Then in the DimProduct (structure, in the solution explorer) you can add newly added tables – use source view pane and add new tables. Next add the following attributes to the dimension:
            1. ProductSubCategoryKey, ProductSubCategoryAlternateKey, EnglishProductSubCategoryName
            2. ProductCategoryKey, ProductCategoryAlternateKey, EnglishProductCategoryName
    c. Please note that, changing the structure of a dimension in a Dimension folder (Solution explorer) only affects the cube, if this dimension structure is actually used by the cube. To see which ones are currently used – you need to go to the Cube Structure pane (in cube view – accessed by right-clicking on the newly created cube and selecting Open; or double-clicking it). There you'll find two major panes that define the contents of the cube – that is Measures and Dimensions. Measures contain all information on measures currently defined in the cube, whereas Dimensions contains all information on the dimensions currently defined in the cube. Note that Date is there 3 times – make sure You understand, why this is the case. As a side note, look at the Dimension Usage pane (right to the Cube Structure) – all details on how dimensions relate to facts, and how they are used, are represented there.
    d. We have also noticed that we are missing a FullName field in the Customer data. The problem here is that such a field was not created during ETL. We can use a simple workaround by creating a named calculation within in the source data view (in the source data view part of the solution explorer). Within context menu of DimCustomer there is an option to create a named calculation – this uses standard SQL expressions (you write native SQL expressions that match the syntax of your underlying database - behind the scenes, SSAS injects this expression as a subquery column when it queries your data source).
            1. Introduce FullName in Customer dimension that concatenates the FirstName, MiddleName, and LastName columns into a single column
            2. Add newly created FullName attribute to the dimension

   e. Create hierarchies by dragging and dropping the attributes from the dimension to hierarchies. Note that, most general attributes are on top, and further down you specify more fine-grained attributes.
      i. Time
         1. DateKey --> Day --> Month --> Quarter --> Semester --> Year
         2. DateKey --> WeekDay --> WeekYear --> Year
6. Deploy and process the cube – to check for any errors.
7. Assure proper formatting and aggregates of measures.
   a. Aggregation This can be done using the AggregateFunction property, which determines how SSAS rolls up data from the lowest grain up to the total/subtotal levels. To set it you need to open the Cube Designer and go to the Cube Structure tab. In the Measures pane, click on the measure you want to configure. In the Properties window (bottom right), look for AggregateFunction. Common Aggregation Types include:
      i. Sum: The default. Adds all values together (e.g., Sales Amount).
      ii. Count / DistinctCount: Counts rows or unique values (e.g., Customer Count). Note: Distinct Count creates a separate internal partition and can impact performance.
      iii. Min / Max: Finds the lowest or highest value (e.g., Max Temperature).
      iv. AverageOfChildren: Useful for rates or averages where you want to average the sub-components.
      v. ByAccount: Uses special rules defined by an Account intelligence dimension (common in financial cubes).
   b. Formatting measures is done using the FormatString property, which dictates how numbers appear to the end-user (e.g., adding currency symbols, commas, or percentages). If left blank, numbers will display as raw, unformatted decimals or integers. To set it you need to in the same Measures pane, click your measure. In the Properties window, locate FormatString. Then select a standard format from the drop-down list or type a custom string:
      i. Currency - Displays based on the server/client regional settings (e.g., $1,234.56).
      ii. Percent - Multiplies the value by 100 and appends % (e.g., 0.15 becomes 15.00%).
      iii. Standard - Adds thousands separators and two decimal places (e.g., 1,234.56).
      iv. #,# - A clean format for whole numbers with thousands separators (e.g., 1,234).
8. Deploy and process the cube – to check for any errors.

Further modify the cube by discretising attributes and introducing new dimensions.

9. Resources: *Automatically Grouping Attribute Members:* https://docs.microsoft.com/en-us/sql/analysis-services/lesson-4-3-automatically-grouping-attribute-members?view=sql-server-2017
10. We need to create a discrete set of list prices, as it is hard for the user to analyze individual and mostly unique prices. Discretization allows grouping a continuous range of numeric values into a discrete set of buckets (intervals), e.g., grouping a Calculated Age column into age brackets like under 20, 20-29, 30-39, etc. To set up discretization, you must configure three specific properties on a dimension attribute. You can find these in the Properties pane after selecting your attribute in the Dimension Designer. Then DiscretizationMethod defines how SSAS will calculate the buckets, and DiscretizationBucketCount defines how many buckets SSAS should create. (If left at 0, SSAS automatically calculates an optimal number of buckets based on the data).
    a. Define discretised attributes in Product dimension:
       i. ListPriceBin as ListPrice
          1. Consider using different grouping methods and different number of bins
11. Finally update hierarchies in the following dimensions:
    a. Customer
       i. FullName --> Gender
    b. Product
       i. ProductKey --> SubCategory --> Category
       ii. ProductKey --> Color
12. Deploy and process the cube – to check for any errors.
13. Introduce new dimension SalesLocation – DimSalesTerritory. Introduce main attributes and create a hierarchy.

14. Deploy and analyse the cube - use Power BI and the built-in browser
15. Save the project – Make sure to save and externally store this project.

TASK 2

Prepare a simple cube based on data structures (star schema) prepared in the previous laboratory assignments (simple ETL for AdventureWorks data):

1. Propose dimensions and their attributes
    a. Define exemplar hierarchy (hierarchies) within each dimension
2. Propose facts and define measures
    a. Define measures and proper aggregation of measures
3. Deploy and process the cube
    a. Browse the data - Check if results are available

Use the prepared data cube perform simple data analysis using MS Excel/PowerBI or some other tool.

4. In separate sheets, please create pivot tables (with charts) for already identified user requirements.
    a. Use user requirements from the previous lab assignments.

As the result, please upload the entire SSAS project folder (use compression) and MS Excel file and screenshots with the results.

TASK 3 (*)

Further modify the cube – introduce attribute relationships and discretise attributes. Use the following tutorials:

a. Sorting attribute members: https://docs.microsoft.com/en-us/sql/analysis-services/lesson-4-5-sorting-attribute-members-based-on-a-secondary-attribute?view=sql-server-2017
b. Specifying Attribute Relationships Between Attributes in a User-Defined Hierarchy: https://docs.microsoft.com/en-us/sql/analysis-services/lesson-4-6-specifying-attribute-relationships-in-user-defined-hierarchy?view=sql-server-2017

to introduce the following changes in the cube from the previous task:

1. Define proper ordering of attribute values in Time dimension
    a. Consider month names, day of the week names, etc.
2. Define necessary Attribute Relationship in Time dimension
    a. Please make sure that the hierarchy is displayed properly (adequate MonthName order, etc.)
3. Process and deploy cube as CubeInternetSales. Verify correctness

TASK 4 (*)

Modify the cube – introduce perspectives, partitions and aggregates. Use the following tutorials:

a. Partitions: https://docs.microsoft.com/en-us/sql/analysis-services/multidimensional-models/partitions-in-multidimensional-models?view=sql-server-2017 and https://docs.microsoft.com/en-us/sql/analysis-services/multidimensional-models/create-and-manage-a-local-partition-analysis-services?view=sql-server-2017
b. Partition storage modes: https://technet.microsoft.com/en-us/library/ms174915.aspx or https://docs.microsoft.com/en-us/sql/analysis-services/multidimensional-models/set-partition-storage-analysis-services-multidimensional?view=sql-server-2017
c. Aggregates design: https://docs.microsoft.com/en-us/sql/analysis-services/multidimensional-models/designing-aggregations-analysis-services-multidimensional?view=sql-server-2017 , https://www.mssqltips.com/sqlservertip/2439/optimize-a-sql-server-analysis-services-measure-group-partition-for-performance/ , https://www.mssqltips.com/sqlservertip/2876/improve-sql-server-analysis-services-performance-with-the-usage-based-optimization-wizard/ or https://www.mssqltips.com/sqlservertip/3406/sql-server-analysis-services-aggregation-designs/

to introduce the following changes in the cube from the previous task:

1. Add New Perspective – In your cube add new perspective representing:

   a. Product Manager Perspective – without access to specific customers and locations, just products and dates (without weekly hierarchy)
   b. Sales Manager Perspective – without access to specific products, just dates, customers and locations
   c. Process and deploy cube. Verify correctness.
2. Add New Partitions:
   a. In your cube add two partitions each representing a different sales year:
      i. One for 2010-2013 (older, cold data) and one for 2014 (fresh, hot data)
   b. Process and deploy cube, check if everything is correct.
3. Define custom aggregations:
   a. In your cube design set-up new aggregates that would allow reaching a performance gain of about 35%. Remember to view the established aggregates and identify which aggregates were created.
   b. Process and deploy cube. Verify correctness

TASK 5 (*)

Further modify the cube – define new calculations and measures:

1. Define new calculation within the cube:
   a. Using Calculations / New Calculated Member add new measure representing Profit. What attributes you should be using?
      i. Use the following expression for the Color Expression / Back color:
         1. *iif ([Measures].[Profit] < 1000000, 255 /*Red*/, 65280 /*Green*/)*
   b. Calculate the shipment cost percentage – as the ratio of freight and sales amount
   c. Calculate the discount percentage – as the ratio of discount amount and sales amount
2. Process and deploy cube as CubeInternetSales
   a. Verify correctness and the new measures (and color expression) using MS Excel
3. Add New Measures - In your cube add new measure representing:
   a. number of customers using Customer dimension – CustomerDCount
   b. number of customers using fact table – CustomerFCount
   c. distinct number of customers who have bought specific products – CustomerDistinct
4. Verify the correctness of the new measures
   d. Check data in the cube browser against proper SQL statements – is it correct?
   e. Prepare a report with number of customers who have bought specific products
   f. Prepare a report with distinct number of customers who have bought specific products
      i. Use dimension usage options
5. Process and deploy cube as CubeInternetSales
   a. Verify correctness

ADDITIONAL CONTENT

- Data Cubes
   a. General Resources - https://docs.microsoft.com/en-us/sql/analysis-services/multidimensional-modeling-adventure-works-tutorial?view=sql-server-2017
      i. Focus on defining Advanced Attribute and Dimension Properties, hiding and Disabling Attribute Hierarchies, defining the Unknown Member and Null Processing Properties, defining Calculations, defining Perspectives, defining Key Performance Indicators (KPIs)
   b. Sorting attribute members:
      i. https://docs.microsoft.com/en-us/sql/analysis-services/lesson-4-5-sorting-attribute-members-based-on-a-secondary-attribute?view=sql-server-2017
   c. Automatically Grouping Attribute Members:
      i. https://docs.microsoft.com/en-us/sql/analysis-services/lesson-4-3-automatically-grouping-attribute-members?view=sql-server-2017
   d. Specifying Attribute Relationships Between Attributes in a User-Defined Hierarchy:
      i. https://docs.microsoft.com/en-us/sql/analysis-services/lesson-4-6-specifying-attribute-relationships-in-user-defined-hierarchy?view=sql-server-2017
- Data Cubes – advanced aspects:

a. Calculations
   i. https://docs.microsoft.com/en-us/sql/analysis-services/multidimensional-models/calculations-in-multidimensional-models?view=sql-server-2014
b. Partitions
   i. https://docs.microsoft.com/en-us/sql/analysis-services/multidimensional-models/partitions-in-multidimensional-models?view=sql-server-2017
   ii. https://docs.microsoft.com/en-us/sql/analysis-services/multidimensional-models/create-and-manage-a-local-partition-analysis-services?view=sql-server-2017
c. Partition storage modes
   i. https://technet.microsoft.com/en-us/library/ms174915.aspx
   ii. https://docs.microsoft.com/en-us/sql/analysis-services/multidimensional-models/set-partition-storage-analysis-services-multidimensional?view=sql-server-2017
d. Aggregates design
   i. https://docs.microsoft.com/en-us/sql/analysis-services/multidimensional-models/designing-aggregations-analysis-services-multidimensional?view=sql-server-2017
   ii. https://www.mssqltips.com/sqlservertip/2439/optimize-a-sql-server-analysis-services-measure-group-partition-for-performance/
   iii. https://www.mssqltips.com/sqlservertip/2876/improve-sql-server-analysis-services-performance-with-the-usage-based-optimization-wizard/
   iv. https://www.mssqltips.com/sqlservertip/3406/sql-server-analysis-services-aggregation-designs/

(*) GX – DATA QUALITY MANAGEMENT AND MONITORING

Great Expectations (GX Core) is the most popular open-source Python library for testing, documenting, and monitoring data quality. It allows you to define "unit tests" specifically for your data. It is important to note that in ETL processes, validation is typically triggered after data is loaded into the staging and/or target tables.

GX is built around several key concepts:

- **Data Context:** Stores the configuration for connections and tests.
- **Expectations:** The rules (tests) you define (e.g., "column X must not have NULL values").
- **Data Assets:** Specific datasets (e.g., the Sales.SalesOrderDetail table) originating from connected data sources.
- **Validation Definitions:** A set of rules linked to specific data.
- **Checkpoints:** The process that runs the validation and generates reports.
- **Data Docs:** Human-readable HTML reports showing test results.

Every GX process follows a common schema—various GX components are defined and utilized at each stage:

1. GX Environment Configuration

The **Data Context** is used to define and run the GX workflow. It is a Python object providing access to configurations, metadata, workflow actions, and data validation results.

2. Establishing Connections and Defining Batches

- **Data Source:** A representation of a data store in GX; it defines how to connect and supports various types, including databases, schemas, and cloud object storage.
- **Data Asset:** A collection of records within a Data Source (e.g., a relational table or a query result).
- **Batch Definition:** Tells GX how to organize records in a Data Asset. An asset can be validated as a single batch or split into multiple batches.

3. Defining Expectations (Data Quality Tests)

- **Expectation:** A verifiable statement about data. Similar to assertions in traditional Python unit tests, expectations provide a flexible, declarative language for describing expected data properties.
- **Expectation Suite:** A collection of expectations used to validate a batch of data, streamlining the validation process. You can apply the same suite to different batches or multiple suites to the same data for different scenarios.

4. Running Validation and Monitoring

- **Validation Definition:** Explicitly pairs a Batch of data with an Expectation Suite.
- **Validation Result:** Returned after validation, indicating how well the data meets expectations.
- **Checkpoint:** The primary means of validating data in a production GX deployment. Checkpoints run a list of Validation Definitions using shared parameters and can trigger automated actions based on results.
- **Actions:** A mechanism to integrate Checkpoints with data pipeline infrastructure. Typical use cases include sending email alerts or Slack/Microsoft Teams notifications.
- **Data Docs:** Human-readable documentation containing Expectation Suite definitions and Validation Results.

INSTALLING THE GX CORE ENVIRONMENT

GX Core is a Python library installable via pip:

```
# Install the main library
pip install great_expectations
```

To work with SQL Server (AdventureWorks), you need additional libraries for connectivity:

```
# Install SQL Server dependencies (SQLAlchemy + driver)
pip install sqlalchemy pyodbc
```

Important: Ensure the **ODBC Driver for SQL Server** is installed on your operating system so pyodbc can connect to the database.

To verify the installation, run the following in your Python interpreter:

```
import great_expectations as gx
print(gx.__version__)
```

CONFIGURING THE SQL SERVER CONNECTION

In GX Core (version 1.0+), we work primarily with objects. Assuming your AdventureWorks database is local:

Initializing Context and Data Source

```
import great_expectations as gx

# 1. Create a temporary or persistent context
context = gx.get_context()

# 2. Define the Connection String
connection_string                                                    =
"mssql+pyodbc://USER:PASSWORD@SERVER/AdventureWorks?driver=ODBC+Drive
r+18+for+SQL+Server"

# 3. Add the Data Source
db_source = context.sources.add_sql(
   name="my_adventure_works_db",
   connection_string=connection_string
)

# 4. Define the Asset (table to monitor)
asset_name = "sales_details"
db_asset          =          db_source.add_table_asset(name=asset_name,
table_name="SalesOrderDetail", schema_name="Sales")

# 5. Define the batch (as the whole table)
batch_definition   =   db_asset.add_batch_definition_whole_table("batch
definition")
batch = batch_definition.get_batch()
```

DEFINING QUALITY RULES (EXPECTATIONS)

Below is an Expectation Suite containing three business rules:

1. **OrderQty** must be greater than 0.

2. **UnitPrice** cannot be empty.

3. **CarrierTrackingNumber** must match specific allowed values.

```
# Create the Expectation Suite
suite_name = "adventure_works_quality_suite"
suite = context.suites.add(gx.ExpectationSuite(name=suite_name))

# Test 1: No NULLs in UnitPrice (Critical severity)
suite.add_expectation(gx.expectations.ExpectColumnValuesToNotBeNull(c
olumn="UnitPrice", severity="critical"))

# Test 2: OrderQty must be positive (Warning severity)
suite.add_expectation(gx.expectations.ExpectColumnValuesToBeBetween(
    column="OrderQty", min_value=1, severity="warning"
))

# Test 3: Specific values in CarrierTrackingNumber
suite.add_expectation(gx.expectations.ExpectColumnValuesToBeInSet(
   column="CarrierTrackingNumber",
   value_set=["4911-403C-98", "4E0A-4F89-AE"],
   severity="warning"
))
```

Running Monitoring (Validation & Checkpoints)

```
# Link data to the suite
validation_def = context.validation_definitions.add(
    gx.ValidationDefinition(
        name="daily_sales_validation",
        data_asset=db_asset,
        suite=suite
    )
)

# Create and run Checkpoint
checkpoint = context.checkpoints.add(
    gx.Checkpoint(
        name="sales_checkpoint",
        validation_definitions=[validation_def],
        result_format="SUMMARY"
    )
)

result = checkpoint.run()

if result.success:
    print("Data is correct!")
else:
    print("Data quality issues detected!")
```

HANDLING ERRORS AND DATA DOCS

To retrieve all rows that failed validation (e.g., for quarantining):

```
from great_expectations.expectations import UnexpectedRowsExpectation

for evr in result.results:
    if not evr.success and isinstance(evr.expectation,
UnexpectedRowsExpectation):
        unexpected_rows = validation_def.get_unexpected_rows(
            evr.expectation,
            batch_parameters=result.batch_parameters,
        )
        print(f"{len(unexpected_rows)} unexpected rows found")
```

To generate and view the HTML report:

```
context.build_data_docs()
context.open_data_docs()
```

WHY IMPLEMENT GX IN ETL?

- **Fail-Fast:** Stop the ETL pipeline if data doesn't meet standards before it reaches the warehouse or Power BI.
- **Automated Documentation:** Easily monitor and maintain business rules applied to the data.
- **SQL Support:** GX doesn't load entire tables into RAM—most validations run as SQL queries directly on the server via SQLAlchemy.